



# APPLICATION DEVELOPMENT PRACTICES

**Dr.Lê Hùng Tiến**  
**VLU**



## **APPLICATION DEVELOPMENT PRACTICES**

# **Analyzing and Fixing Defects**



# Director's Overview

---

- Characteristics of Software Quality
- What is a Defect?
- Cost of Defects
- Defects as Opportunities
- Tips for Finding Defects
- Tips for Fixing Defects
- Defect Distribution
- Estimating Defect Fixes



# Characteristics of Software Quality

---

- Correctness
    - Defects
  - Usability
  - Performance
  - Scalability
  - Extensibility
  - Installability
  - Maintainability\*
  - Portability\*
  - Reusability\*
  - Readability\*
  - Testability\*
  - Accessibility
  - Auditability
  - Configurability
    - Personalization
    - Internationalization
  - Efficiency
  - Interoperability
  - Operational availability
  - Flexibility\*
  - Reliability
  - Robustness
  - Safety
  - Security
- \* Internal software characteristic



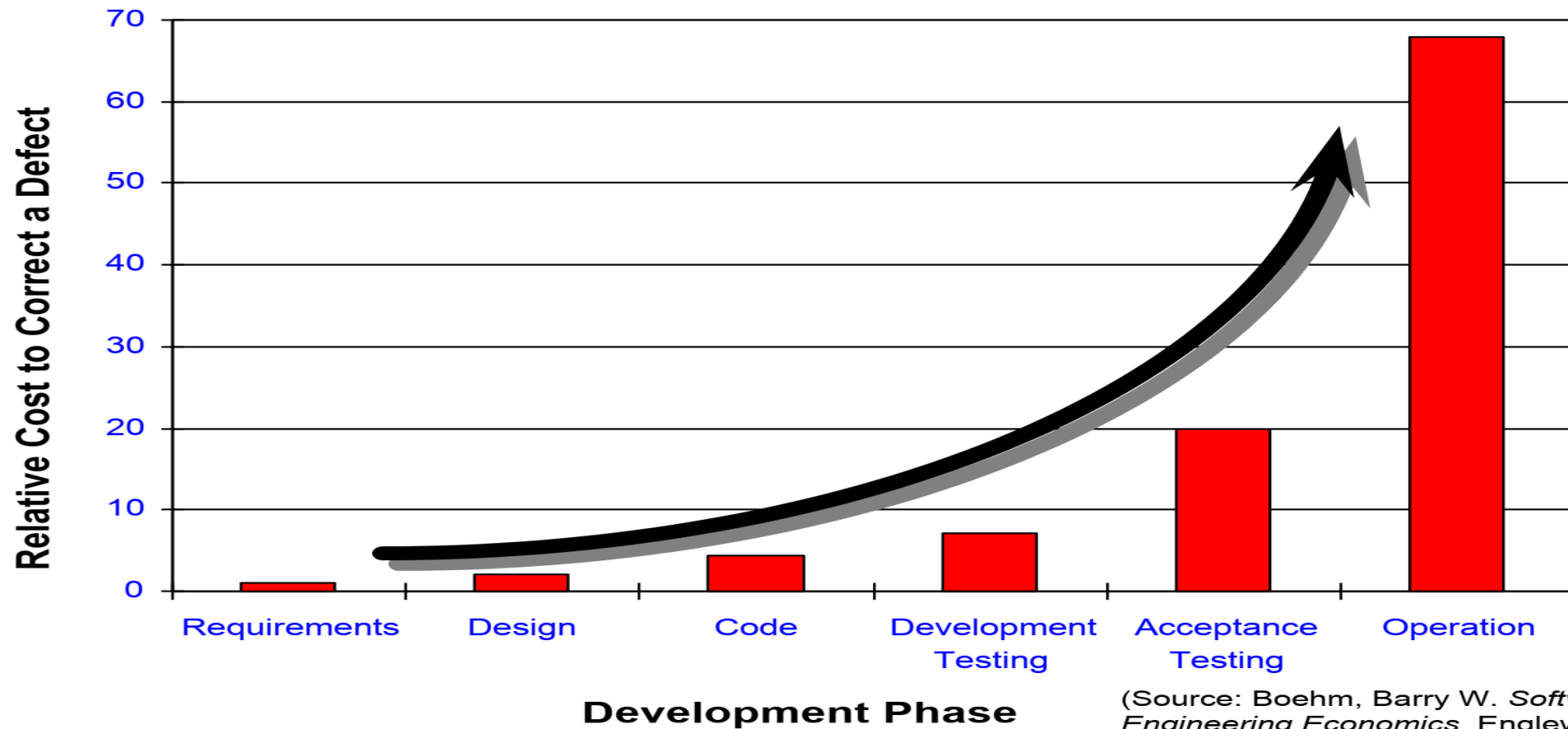
# What is a Defect?

---

- Code defects are programming errors
- Sources of defects:
  - Poor understanding of requirements
  - Poor design
  - Poor coding practices
  - Limited or no unit and integration testing
  - Introduced during defect fixing
  - Typos
  - Others ...



# Relative Cost of Fixing a Defect



(Source: Boehm, Barry W. *Software Engineering Economics*. Englewood Cliffs, NJ: Prentice-Hall, 1981)





# Defects & Schedules

---

- “[IBM] found that products with the lowest defect counts were also the products with the shortest schedules”
- If you’re finding more than 5% of your defects after product release:
  - Vulnerable to low quality problems
  - Taking more time to develop than necessary
- If you can prevent defects, or detect and remove them early, you can realize a significant schedule benefit



# Defects as Opportunities

---

- Assuming that the developer does not want to have defects ...  
... it means that the developer doesn't fully understand:
  - What the software does
  - How the software works
  - How to make the software do the right things
- Intentional development?  
or
- Developing by trial and error?

(Source: McConnell, Steve *Code Complete*. Microsoft Press, 2004. pages 537-538)





# Defects as Opportunities

---

- As a developer you can:
  - Learn about the program you're working on
    - Do you fully understand how it all works?
  - Learn about the kind of mistakes you make
    - Why did you make it?
    - How could you have found it quickly?
    - How could you have prevented it?
  - Learn about the quality of your code (from the perspective of someone who has to read it)
    - Is your code easy to read?
    - How could it be better (ask someone)?



# Defects as Opportunities

---

- As a developer you can:
  - Learn about how you solve problems
    - Does your approach to finding problems work?
    - Do you find defects quickly?
    - Do you guess at where the problem is?
  - Learn about how you fix defects
    - Do you make a complete problem diagnosis and implement systematic corrections?



# Steps for Fixing Problems

---

1. Identify how to repeat the problem
  - Gather data through repeatable experiments
  - You may need to replicate the user's machine
2. Form a hypothesis that accounts for the behavior
3. Design an experiment that proves/disproves the hypothesis
  - By testing the software or examining the code
  - Run the experiment
  - Repeat as needed
4. Fix the defect
5. Test the fix
6. Look for similar problems

(Source: McConnell, Steve *Code Complete*.  
Microsoft Press, 2004. pages 540-541)



# Tips for Finding Defects

---

- A software problem may result from one or more defects
- If you are having trouble finding the defects associated with a problem:
  - Use all data available to make your hypothesis
    - Don't discard data. Ask why it doesn't fit and create a new hypothesis
  - Refine the test cases that produce the defect
    - If all your test cases are passing, you need to refine them until you can reproduce the failure
  - Isolate the problem
    - Defects are easier to find in smaller fragments of code

(Source: McConnell, Steve *Code Complete*. Microsoft Press, 2004. pages 540-541)



# Tips for Finding Defects

---

- If you are having trouble finding the defects associated with a problem:
  - Use available tools
  - Reproduce the problem several different ways
  - Generate more data to generate more hypotheses
  - Use the results of negative tests
  - Brainstorm for possible hypotheses
    - Don't limit your focus to the first hypothesis that you think of
  - Make a list of things to try
    - Don't spend too much time going down a box canyon

(Source: McConnell, Steve *Code Complete*. Microsoft Press, 2004. pages 540-548)





# Tips for Finding Defects

---

- If you are having trouble finding the defects associated with a problem:
  - Isolate the suspicious regions of code
    - Remove parts of the product from your testing. If the problem stays, it wasn't in the part you removed
  - Be suspicious of classes and methods that have had defects before
    - Keep data on defects found in classes and methods
  - Check code that has changed recently
  - Integrate incrementally
    - Check for defects by adding new pieces of the system one at a time. If you find a new problem, remove the new piece and test it separately

(Source: McConnell, Steve *Code Complete*. Microsoft Press, 2004. pages 540-548)





# Tips for Finding Defects

---

- If you are having trouble finding the defects associated with a problem:
  - Check for common defects
    - Use code-quality checklists to stimulate your thinking
  - Talk to someone else about the problem
    - Often you will uncover the defect just by describing what you have done so far to someone else
  - Take a break from the problem
    - Go for a walk. Work on something else. Let your subconscious work on the problem



# Know Your Available Tools

---

- Scientific method
  - Isolate the problem and make it repeatable
- Compilers, memory & syntax checkers
- Interactive Debugger
  - Determine the behavior of your software as it executes
  - Breakpoints
- Print statements
  - To display key software events and variables



# Know Your Available Tools

---

- Internal trace log
  - Logs system events in the order that they occur, along with relevant data
- In-circuit emulator (ICE)
  - A hardware device used to debug the software of an embedded system
- Design review
  - Walk through the broken code with many eyes



# Tips for Fixing Defects

---

- Understand the problem before you fix it
- Understand the program, not just the problem
  - Or at least the vicinity of the problem
- Confirm the defect diagnosis
  - Run test cases that prove your hypothesis
- Don't rush
  - Time pressure can lead to rushed judgment, incomplete diagnosis and incomplete correction



# Tips for Fixing Defects

---

- Fix the problem, and not the symptom
  - Don't put in special case fixes. They are hard to maintain, and probably don't solve the problem
- Look for similar defects
- Don't change code randomly and hope it fixes things
- Fix one problem at a time
  - Saving up changes and checking them all at the same time just makes it more difficult for someone else to figure out what fixed what later



# Tips for Fixing Defects

---

- Check your work
  - Consider asking someone else to check it as well
- Add unit tests that verify that the problem has been thoroughly fixed
- Document the fix
  - Enter the bug number and describe what was fixed
  - Capture root cause data in the change request system
- Check your tests
  - They can have defects too

(Source: McConnell, Steve *Code Complete*.  
Microsoft Press, 2004. pages 540-548)





# Defect Distribution

---

- Defects are not distributed evenly throughout the source code
  - Specific classes will be error prone. Look for these
  - Highly defective routines are very expensive
- From studies;
  - 80% of defects are found in 20% of classes or routines
  - 50% of defects are found in 5% of classes



# Estimating Defect Fixes

---

- Most defects are easy to fix
  - ~85% can be fixed in a few hours
  - ~15% can be fixed in 4 hours to a few days
  - ~1% take longer than a few days
- No matter what management wants to hear, you can't provide an estimate for fixing a problem until you have diagnosed the problem
  - Some take a long time to diagnose and are easy to fix
  - Some are easy to diagnose and are hard to fix
- Pressure to fix defects quickly can easily cause the insertion of new defects



# Estimating Defect Fixes

---

- Don't assign junior programmers to fix defects
  - They can unintentionally introduce new defects
- If a defect must be fixed quickly, consider assigning 2 or 3 developers to it



# Rules for Making Code Changes

---

- Developers responsible for:
  - Keeping their machine in synch with the configuration management system
  - Fixing one defect per set of source file revisions
  - Creating/updating automated unit tests
    - May want to consider test first programming style
  - Creating/updating automated system and/or acceptance tests
  - Ensuring that all tests run successfully before integrating code back into master repository
  - Resolving conflicts when checking code back in
  - Updating change documentation



# Summary

---

- As a developer YOU are responsible for creating high quality code in a timely and predictable manner
- Learn:
  - The software you are working on
  - Coding and problem solving techniques
  - The development tools available to you
- Share your knowledge, and learn from your peers
- In most cases, the projects with the fewest defects have the shortest schedules